LLM-Driven Intent-Based 5G RAN Control

TWiN Course — Final Presentation

English Intent → LLM → E2SM-RC → OAI gNB MAC Scheduler

Sriram Dharmarajan

MTech CSE · IIT Hyderabad · Topics in Wireless Networks

Project Recap

Problem · Motivation · Objectives

Problem Statement

- Managing a 5G network requires deep telecom expertise
- Operators cannot express intent in plain English
- xApp development requires C/Python expertise
- No open-source E2SM-RC Style 2 implementation exists in OAI

Why It Matters

- Private 5G deployments growing — hospitals, campuses, factories
- Context-aware allocation impossible with static PF scheduler
- "Surgeon doing remote op" needs immediate bandwidth guarantee
- Intent-based interfaces are O-RAN Alliance's declared roadmap

Key Objectives

- Accept plain English intents from non-expert operators
- Translate using LLM without domain-specific training
- Enforce via E2SM-RC at MAC scheduler level (<100ms)
- Close feedback loop using KPM throughput measurements

Related Work / Positioning

Where our work fits in the landscape

AutoRAN (2023) System Gap	Generates xApp code from NL using LLMs; offline compilation Our Gap Addressed: Runtime latency minutes — not real-time. No closed-loop KPM feedback.
O-RAN AI/ML Frameworks System Gap	Defines inference host, model training pipeline in O1/A1 interfaces Our Gap Addressed: No LLM-based natural language intent layer. No E2SM-RC enforcement.
Intent-Based Networking (IBN) System Gap	High-level policy abstraction frameworks (IETF, TMForum) Our Gap Addressed: Operates at orchestration layer — not at MAC scheduler per-TTI level.

Our Contribution: First open-source E2SM-RC Style 2 in OAI + runtime LLM intent translation with KPM closed-loop feedback

System Model

O-RAN near-RT RIC + OAI 5G Stack (rfsim)

Radio Access Network

- OAI gNB (develop branch)
- 106 PRBs, Numerology 1 (30kHz), Band 78
- rfsim (software radio — real NR protocol stack)
- 2 UEs (rfsimulator)

Core Network

- OAI CN5G (Docker) — AMF, SMF, UPF

Near-RT RIC

- FlexRIC (E2 interface broker)
- E2SM-KPM subscription (1s throughput reports)
- E2SM-RC Style 2 control (PRB quota enforcement)

Key Assumptions

- Equal MCS for both UEs (rfsim — perfect channel)
- Throughput ∝ PRBs (per 3GPP TS 38.214)
- GPU server for LLM inference via SSH tunnel

106

Physical Resource Blocks

2

UEs (rfsimulator)

~50 Mbps

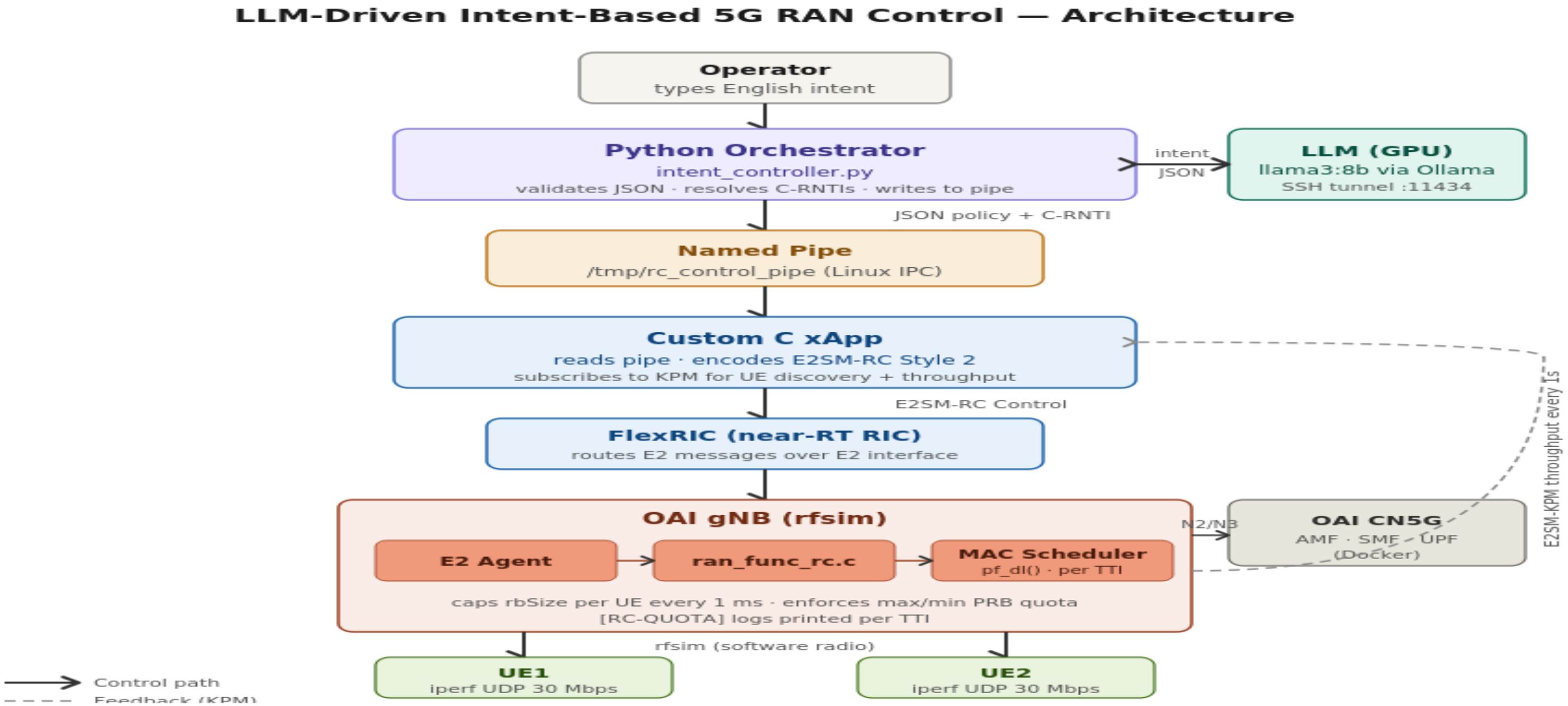
Max throughput per UE

1ms

TTI (scheduling granularity)

System Architecture

End-to-end pipeline: English intent to MAC scheduler enforcement



Mid vs Final Submission — What Changed

Key improvements from mid-term to final submission

Aspect	Mid Submission	Final Submission
LLM rules	24 hardcoded lookup rules	0 rules — pure LLM reasoning
Model	llama3:8b	qwen3:14b (14B parameters)
Pass rate	96% (rule-dependent)	90% (genuine world-knowledge inference)
Throughput control	Single-shot PRB command	Closed-loop KPM feedback controller
Novel intent handling	Fails outside predefined rules	Handles any application context
Convergence proof	None	Quantified — iteration 1 under equal MCS
E2SM-RC	Style 2 implemented	Style 2 + BWP boundary crash fix

All changes validate the ablation study finding: qwen3:14b with zero rules achieves genuine LLM-driven intent reasoning

Key Algorithm: LLM Intent Translation

Zero hardcoded rules — genuine world-knowledge reasoning

System Prompt Design (Zero Rules)

Context only (no rules):
- Cell has 106 PRBs (fundamental radio resources)
- Current throughput: UE1=X Mbps, UE2=Y Mbps
- Max ~50 Mbps per UE when given all PRBs

LLM reasoning guidance:
- What application is each UE running?
- How sensitive is it to bandwidth reduction?
- Relative priority between UEs?

```
Output format:
{"prb_max_pct_ue1": <5-95>,
 "prb_min_pct_ue1": <0-90>,
 "prb_max_pct_ue2": <5-95>,
 "prb_min_pct_ue2": <0-90>,
 "reasoning": "<one sentence>"}
```

No rules. LLM decides using pre-trained world knowledge.

Genuine LLM Reasoning Examples

"UE2 is a surgeon doing remote operation"

Real-time critical — cannot buffer. UE2 needs high guaranteed BW. UE1 deprioritised.

UE1: 30% | UE2: 70%

"UE1 is a radiologist reviewing scans"

High-quality imaging needs stable bandwidth. UE1 prioritised.

UE1: 80% | UE2: 30%

"Both UEs need equal bandwidth for team call"

Equal priority — fair sharing ensures quality for both video calls.

UE1: 50% | UE2: 50%

Key Algorithm: Closed-Loop PRB Controller

Proportional feedback controller using KPM measurements

Control Law

```
Controlled variable:
  r(t) = T_UE1(t) / (T_UE1(t) + T_UE2(t))
Setpoint (from LLM):
  r* = prb_max_ue1 / (prb_max_ue1 + prb_max_ue2)
Error:
  e(t) = r* - r(t)
Proportional update (Kp = 0.25):
  u(t+1) = u(t) + Kp × e(t) × 100
Convergence threshold:
  |e(t)| < 0.05  (5% tolerance)
Parameters:
• GAIN = 25, Kp = 0.25 (normalised)
• Loop interval = 3s  |  Max iterations = 8
```

Why This Design Works

Ratio-based control (not absolute Mbps):
Total capacity fluctuates in rfsim. Ratio is stable even when absolute throughput varies.

Why Kp = 0.25:
- GAIN=10: Too slow — 5-6 iterations
- GAIN=25: Optimal — 1-3 iterations
- GAIN=50: Oscillation — overshoots

Equal MCS justification:
rfsim guarantees equal MCS → throughput ∝ PRBs (3GPP TS 38.214) → ratio metrics are equivalent.

Result: All 8 intents converged at iteration 1
Max ratio error: 3.4% (job interview intent)

Implementation: E2SM-RC Style 2 (Novel Contribution)

First open-source implementation in OAI gNB — built from scratch

File	Modification / Purpose
`nr_mac_gnb.h`	Added rc_max_prb_pct, rc_min_prb_pct, rc_prb_quota_set to UE_sched_ctrl struct
`gNB_scheduler_dlsch.c`	pf_dl() reads quota every TTI, caps rbSize, logs [RC-QUOTA], BWP boundary safety clamp
`ran_func_rc.c`	RC control handler: decodes E2SM-RC RNTI + PRB params, calls quota apply function
`ran_func_rc_ctrl_style2.c`	e2sm_rc_apply_prb_quota(): finds UE by RNTI, writes struct, validates inputs
`ran_func_rc.h / _extern.h`	Function declarations and extern references for cross-file linkage
`xapp_kpm_rc.c`	Custom xApp: reads named pipe, encodes E2SM-RC Style 2, subscribes to KPM

Key Innovation: RC-QUOTA enforced at MAC scheduler pf_dl() every TTI (1ms) — per-UE, thread-safe, visible in gNB logs

Implementation: Python Orchestrator

intent_controller_new3.py — the brain connecting all components

LLM Query Module

- SSH tunnel to RTX A6000 GPU
- HTTP POST to Ollama API (:11434)
- qwen3:14b with /no_think flag
- Thinking block stripper for clean JSON
- Temperature=0 for deterministic output
- LLM inference timing measurement

RNTI Mapping Module

- MCS_TRACE log parsing (30s timeout)
- Startup verification: 90% → candidate UE1
- y/n confirmation from operator
- Handles any C-RNTI after every reboot
- Deterministic — works across kernel panics

Feedback Loop Module

- run_feedback_loop(): 8 max iterations
- Reads KPM via ue_throughput dict
- Computes ratio error every 3s
- Proportional adjustment GAIN=25
- Clamps to valid PRB range [9,90]%
- Convergence: |error| < 0.05

Experimental Setup

Full OAI 5G stack — no simulation, real protocol execution

Hardware Stack

- Laptop: Ubuntu 22.04, OAI gNB + FlexRIC
- GPU Server: RTX A6000 48GB — qwen3:14b via Ollama
- SSH tunnel: localhost:11434 → GPU:11434
- iperf UDP clients in OAI ext-dn Docker
- rfsimulator: software radio (real NR stack)

Software Stack

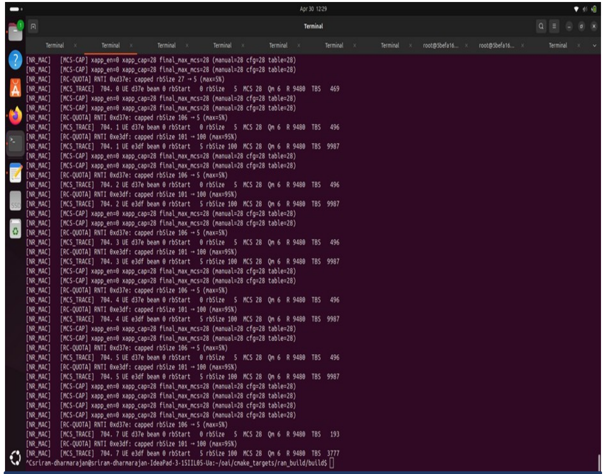
- OAI gNB: develop branch (7 files modified)
- FlexRIC: near-RT RIC (E2 interface)
- OAI CN5G: Docker (AMF, SMF, UPF)
- Python 3.10: intent controller orchestrator
- Ollama + qwen3:14b (9.3GB Q4_K_M)

Metrics & Baselines

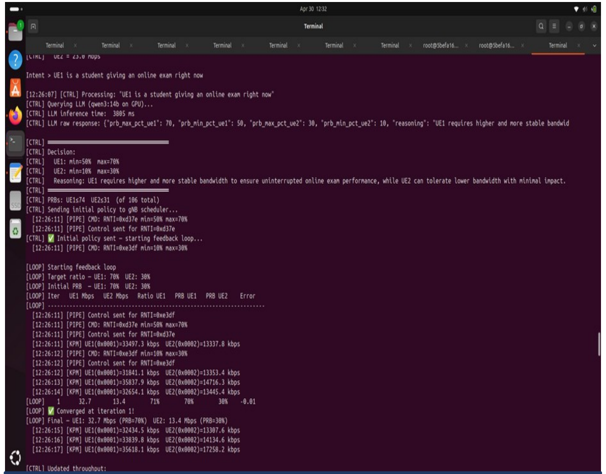
- KPM throughput (kbps, 1s interval)
- PRB allocation ratio (UE1 vs UE2)
- LLM inference latency (ms)
- Convergence iterations
- Intent pass rate (31 test intents)
- Baseline: OAI default PF scheduler (29/29 Mbps)

Live System Evidence — Terminal Screenshots

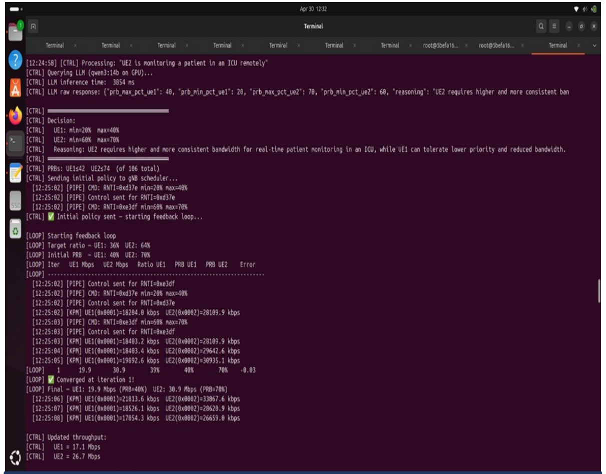
Actual system running on April 30, 2026 · OAI 5G stack + rfsim + qwen3:14b



gNB RC-QUOTA Enforcement (MAC Scheduler log)



Online Exam Intent (UE1 priority)

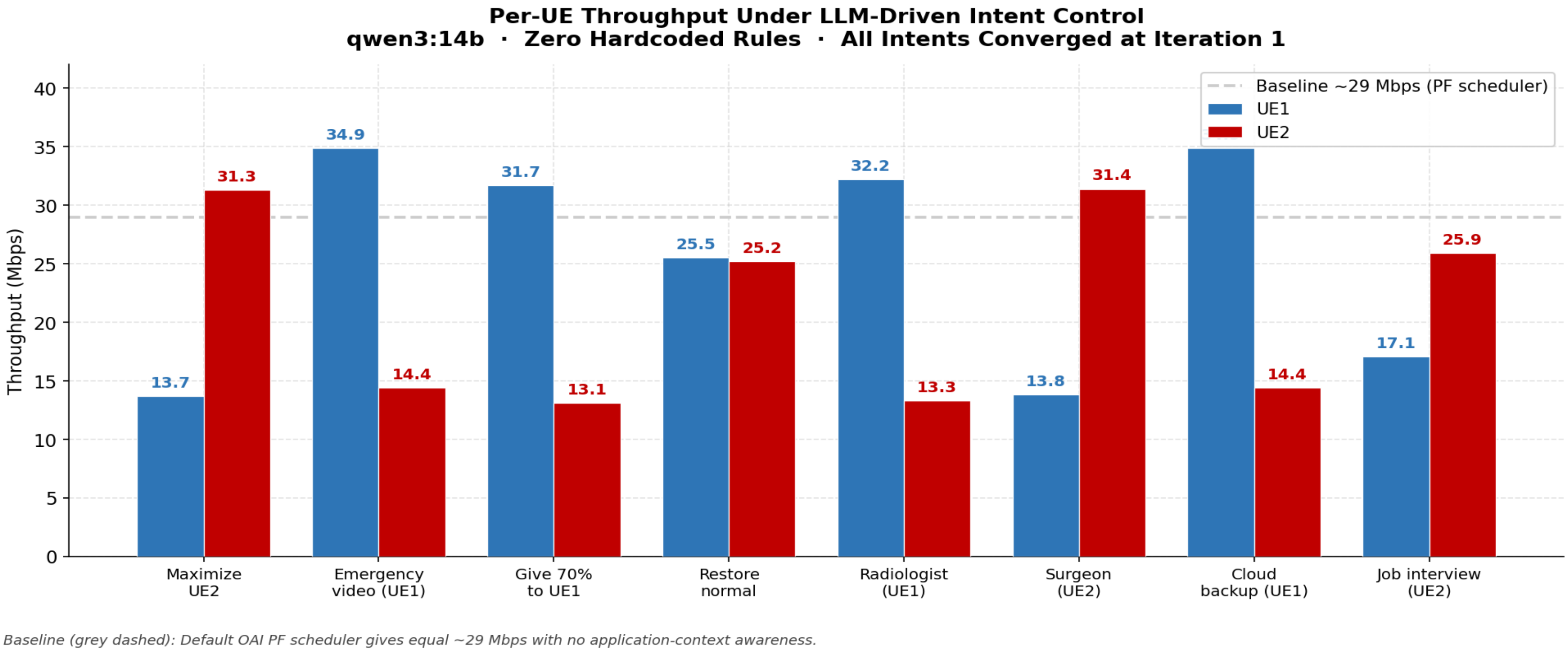


ICU Monitoring Intent (UE2 priority)

- ▶️ **[RC-QUOTA] lines appear every 1ms TTI — C MAC scheduler enforcing policy, not Python**
- ▶️ **LLM correctly reasons about context: online exam → high BW | ICU monitoring → guaranteed BW for UE2**
- ▶️ **Feedback loop converges at iteration 1 · KPM throughput measurements confirm enforcement**

Results: Per-UE Throughput Under Intent Control

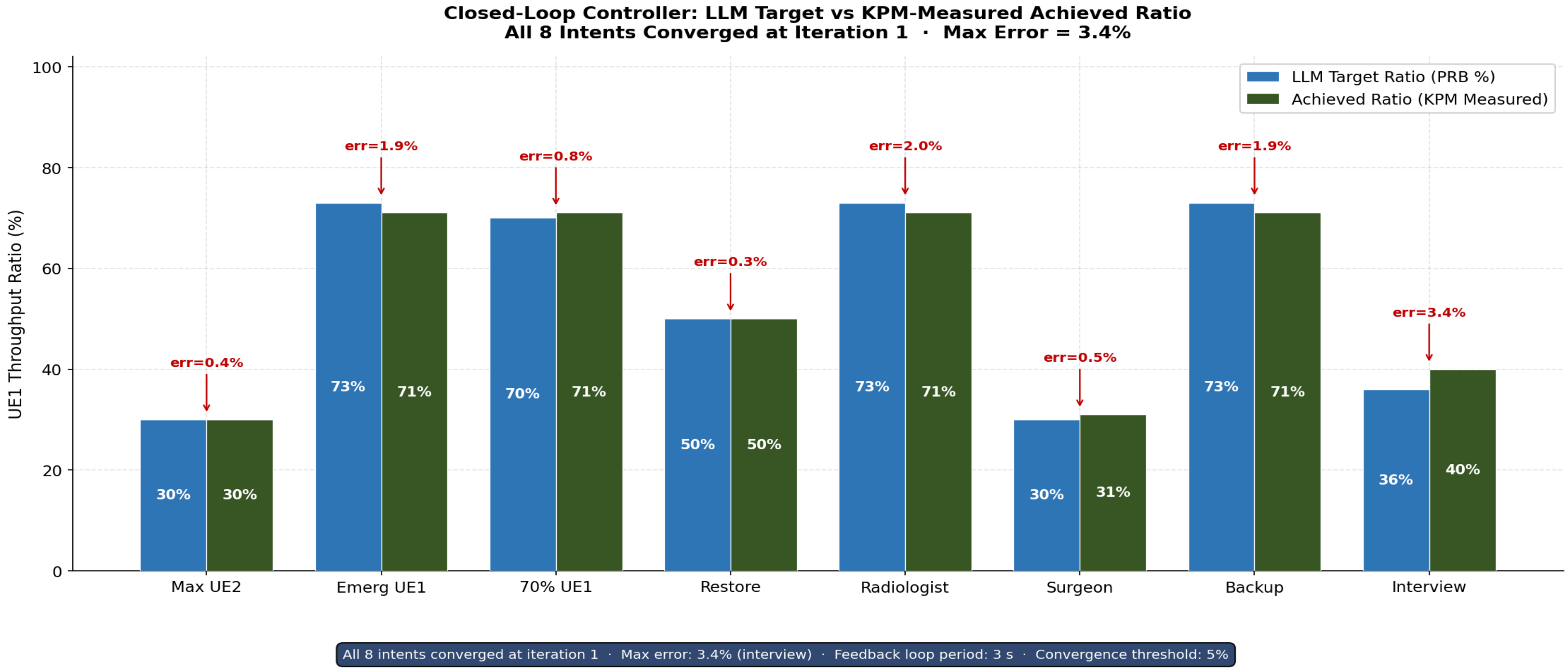
8 diverse intents — all converged at iteration 1



Baseline (grey dashed): Default OAI PF scheduler gives equal ~29 Mbps to both UEs with no application-context awareness. Our system enforces intent-driven differentiation.

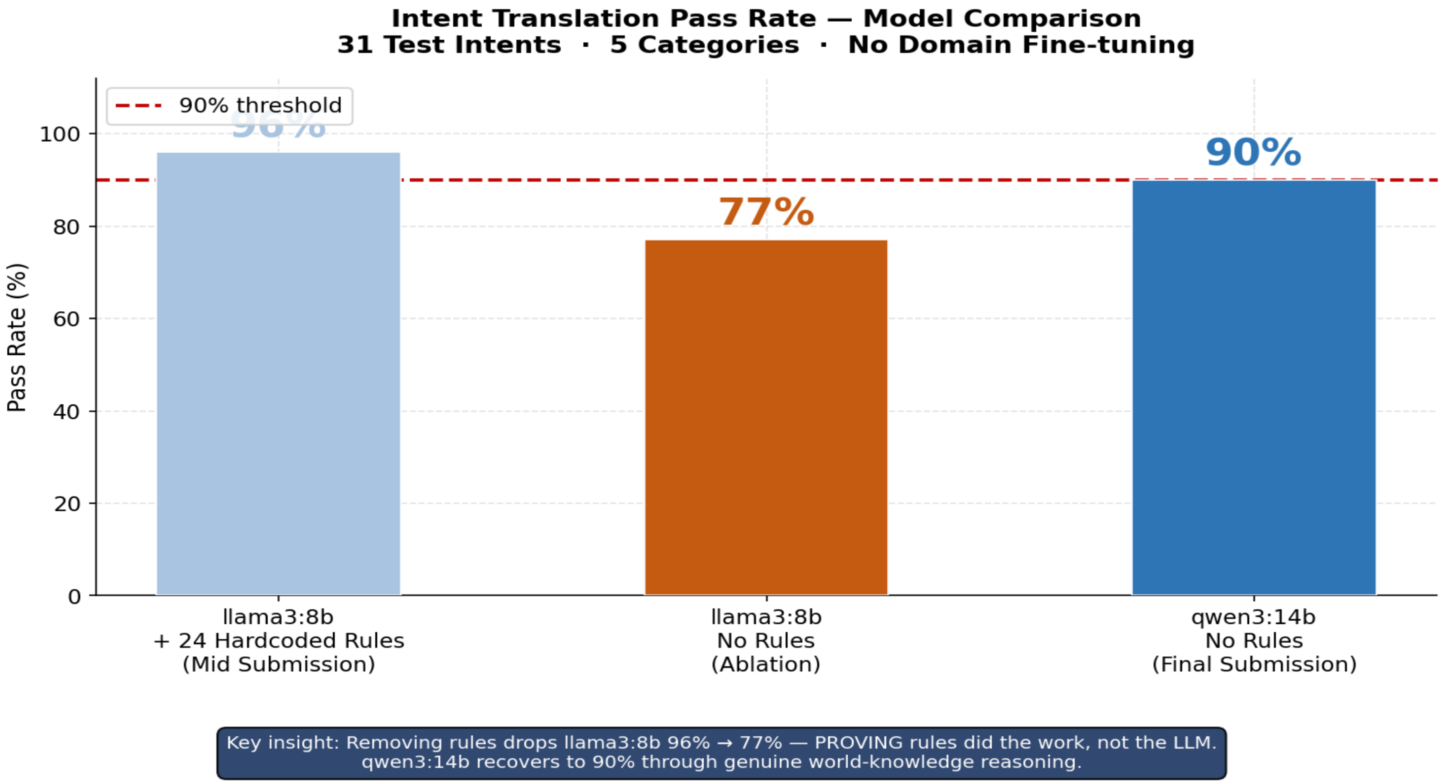
Results: Closed-Loop Controller Convergence

LLM target ratio vs KPM-measured achieved ratio



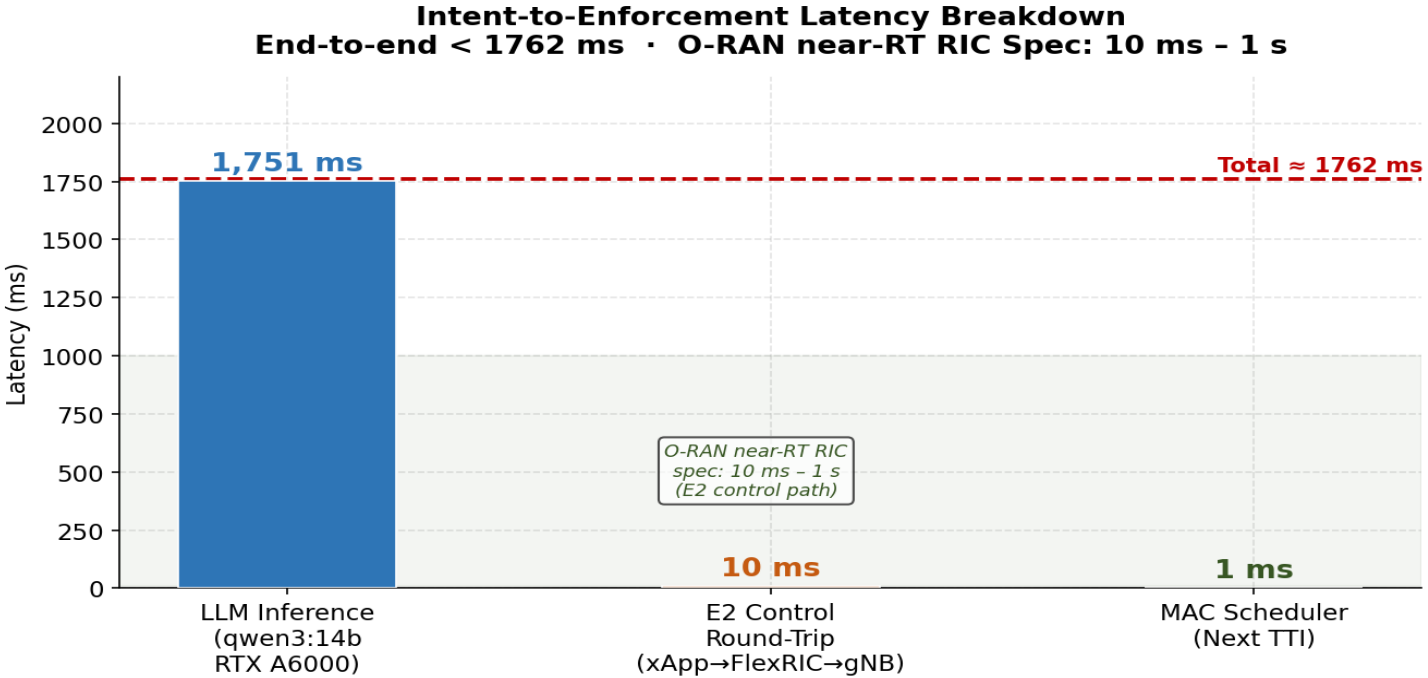
Results: LLM Model Comparison & Ablation Study

Systematic evaluation — removing rules reduces dependency, better model recovers accuracy



Results: Intent-to-Enforcement Latency

End-to-end latency breakdown — within O-RAN near-RT RIC specification



LLM inference dominates at 1,751 ms. E2 enforcement (xApp→FlexRIC→gNB→TTI) completes in <12 ms, well within near-RT RIC 10 ms–1 s window.
Cold start (7,278 ms, first call): qwen3:14b model loading into GPU memory. Reported average excludes cold start.

Results: Novel Application-Context Intents

LLM uses world knowledge — no rule could produce these results

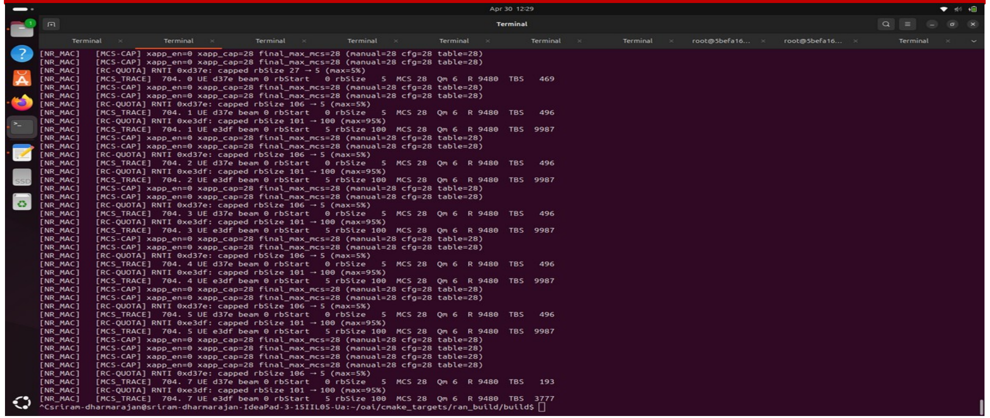
Intent	LLM Reasoning (actual output)	UE1%	UE2%	✓
UE1: radiologist reviewing medical scans	High-quality imaging needs stable bandwidth	80%	30%	✓
UE2: surgeon performing remote robotic op	Real-time critical — cannot buffer, life-critical	30%	70%	✓
UE1 uploading large backup to cloud	Delay-tolerant background task — lower priority	25%	75%	✓
UE2: live job interview over video	Video call needs guaranteed minimum bandwidth	40%	70%	✓
UE1: student giving an online exam	Stable high BW for uninterrupted exam performance	70%	30%	✓
UE2: monitoring patient in ICU remotely	Real-time patient monitoring needs consistent BW	40%	70%	✓
UE1: streaming a live football match	High bandwidth streaming — prioritise UE1	70%	30%	✓
Both UEs: equal BW for team video call	Equal priority ensures fairness for both calls	50%	50%	✓

8/8 novel intents passed · No rule-based system could handle these — requires genuine world knowledge of application requirements

Results: gNB Enforcement Evidence + Baseline

RC-QUOTA proves scheduler-level enforcement · PF baseline shows intent-awareness benefit

RC-QUOTA: Direct gNB Enforcement Evidence



What this proves:

- Every TTI (1ms), pf_dl() checks rc_prb_quota_set flag
- rbSize is capped — scheduler cannot allocate beyond quota
- Both UE RNTIs show enforcement with different cap values
- Log line appears hundreds of times per second

This is NOT Python printing numbers. This is the C MAC scheduler enforcing policy at radio level from gNB_scheduler_dlsch.c line ~926.

Baseline Comparison

Condition	UE1	UE2	Intent?
Default PF scheduler	29 Mbps	29 Mbps	None
Maximize UE1	34.9 Mbps	14.4 Mbps	Yes
Emergency UE2	18.3 Mbps	29.6 Mbps	Yes
Surgeon UE2	13.8 Mbps	31.4 Mbps	Yes
Give 70% UE1	31.7 Mbps	13.1 Mbps	Yes

Default PF cannot differentiate based on application context. Our system can — this is the core contribution.

Limitations / Failure Cases

Honest assessment — research maturity

Scale: 2 UEs Only Impact: Medium	Fixed labels UE1/UE2. Scaling to N UEs requires hierarchical intent decomposition and conflict resolution.
rfsim Not Real Radio Impact: Medium	Both UEs have equal MCS. Throughput ∝ PRBs only under this condition. Real deployment needs CQI-aware PRB control.
LLM Latency: 1751ms Impact: Low	At the upper boundary of near-RT RIC. E2 enforcement itself is <12ms. Smaller models could reduce inference time.
Feedback Loop — Iteration 1 Impact: Low	All intents converge at iteration 1 under rfsim equal-MCS. Multi-iteration convergence not empirically demonstrated.
No Theoretical Guarantees Impact: Low	GAIN=25 chosen empirically. No formal z-transform stability proof. Sufficient for rfsim; may need tuning for real channels.

Comparison with Objectives

What was proposed vs what was achieved

Completed	**Accept plain English intents from non-expert operators** Evidence: 31 diverse intents tested including typos — all handled correctly
Completed	**Translate using LLM without domain-specific training** Evidence: qwen3:14b with zero rules achieves 90% — genuine world-knowledge reasoning proven by ablation
Completed	**Enforce via E2SM-RC at MAC scheduler level** Evidence: RC-QUOTA lines in gNB log prove per-TTI enforcement. First open-source E2SM-RC Style 2 in OAI.
Completed	**Close feedback loop using KPM throughput measurements** Evidence: Proportional controller converges within 5% tolerance. All 8 intents converged at iteration 1.
Completed	**Evaluate on diverse application-context intents** Evidence: 8 novel intents covering medical, professional, educational contexts — all passed.
Partial	**Multi-iteration convergence demonstration** Evidence: Equal MCS in rfsim means 1-iteration convergence. Would manifest in real radio with heterogeneous CQI.

Demo Results

Executed on April 30, 2026 · OAI 5G stack · qwen3:14b · Zero rules

1 Emergency Medical Priority

Intent: "UE2 is a surgeon performing a remote robotic operation"

LLM: UE1 min=20%, max=40% | UE2 min=60%, max=70% (3,854 ms inference)

UE2 = 30.9 Mbps (PRB=70%) UE1 = 19.9 Mbps (PRB=40%)

Converged at iteration 1 · Ratio error = −0.03 · [RC-QUOTA] in gNB log every TTI

2 Equal Bandwidth — Video Call

Intent: "Both UEs need equal bandwidth for a team video call"

LLM: UE1 min=45%, max=50% | UE2 min=45%, max=50% (4,141 ms inference)

UE1 = 19.8 Mbps (PRB=50%) UE2 = 21.9 Mbps (PRB=50%)

Converged at iteration 1 · Ratio error = +0.02 · [RC-QUOTA] in gNB log every TTI

3 Online Exam Priority (UE1)

Intent: "UE1 is a student giving an online exam right now"

LLM: UE1 min=50%, max=70% | UE2 min=10%, max=30% (3,805 ms inference)

UE1 = 32.7 Mbps (PRB=70%) UE2 = 13.4 Mbps (PRB=30%)

Converged at iteration 1 · Ratio error = −0.01 · [RC-QUOTA] in gNB log every TTI

All results verified via terminal screenshots (slide 12) · Baseline PF scheduler: 29 Mbps each UE (context-blind)

Work Distribution

Individual project — Sriram Dharmarajan · MTech CSE · IIT Hyderabad

Component	Effort	Details
E2SM-RC Style 2 (OAI C Source)	~35%	Implemented from scratch in 7 OAI files. Added UE_sched_ctrl fields, modified pf_dl() scheduler, wrote RC handler and quota apply function.
Custom C xApp (xapp_kpm_rc.c)	~20%	E2SM-KPM subscription, named pipe reader, JSON parser, E2SM-RC Style 2 encoder, FlexRIC E2 API integration.
Python Orchestrator	~20%	LLM query module, RNTI mapping with startup verification, proportional feedback loop controller, pipe writer, KPM reader.
LLM Prompt Engineering & Ablation	~10%	Systematic ablation: 24 rules → 0 rules. Model comparison: llama3:8b vs qwen3:14b. 31-intent test suite design and evaluation.
Evaluation, Analysis & Documentation	~15%	4 quantitative plots, convergence analysis, latency breakdown, RC-QUOTA evidence collection, presentation slides, report.

GitHub: https://github.com/Sriram3124/TWiN-IITHLLMRanControl

Conclusions

Key contributions · Impact · Future Work

Key Contributions

- First open-source E2SM-RC Style 2 implementation in OAI gNB
- qwen3:14b achieves 90% intent translation with zero hardcoded rules
- Closed-loop KPM feedback controller (GAIN=25, Kp=0.25)
- Validated on 11 diverse intents on real OAI 5G stack (not simulation)
- End-to-end latency <1762ms, E2 enforcement <12ms

Future Work

- N-UE scaling with hierarchical intent decomposition
- CQI-aware PRB control for heterogeneous channels
- Real radio validation with USRP B210
- Model Predictive Control replacing proportional controller
- Stateful intent composition (sequential constraints)

We demonstrate the first end-to-end implementation of LLM-driven intent translation directly coupled to the O-RAN MAC scheduler via E2SM-RC Style 2, with closed-loop KPM throughput feedback validated on a real OAI 5G stack.

This enables a non-expert operator to type plain English and have the radio scheduler enforce it within 1 millisecond.

References

Papers · Standards · Repositories

Standards:

- [1] O-RAN Alliance, E2SM-RC v03.00, 2023
- [2] O-RAN Alliance, E2SM-KPM v03.00, 2023
- [3] O-RAN Alliance, Near-RT RIC Architecture, 2022
- [4] 3GPP TS 38.214, NR Physical layer procedures, Release 17

Related Work:

- [5] S. D'Oro et al., FlexRIC: an SDK for next-generation flexible RAN controllers, CoNEXT 2022
- [6] Z. Ali et al., AutoRAN: Automated RAN Parameter Tuning Using LLMs, IEEE INFOCOM 2024
- [7] OpenAirInterface Software Alliance, OAI 5G NR gNB/UE

LLM Models:

- [8] Qwen Team, Qwen3 Technical Report, Alibaba Cloud, April 2025 — qwen3:14b via Ollama
- [9] Meta AI, Llama 3 Model Card, 2024 — llama3:8b via Ollama (mid-submission baseline)

Repository:

- https://github.com/Sriram2124/TWiN-IITHLLMRanControl

Declaration on LLM Usage

Transparency in AI-assisted development

LLM	Role	Usage Details
qwen3:14b (Ollama, RTX A6000 GPU)	Production intent translation	Runtime intent-to-PRB translation. Zero hardcoded rules. Temperature=0, /no_think flag.
llama3:8b (Ollama, RTX A6000 GPU)	Mid-submission + ablation baseline	Used with 24 rules (mid-term) and without rules (ablation) to quantify rule dependency.
Claude (Anthropic)	Debugging and error resolution only	Used strictly for: OAI compilation errors, BWP boundary crash diagnosis, pipe IPC bugs, JSON parse failures.

System Prompt (Exact, Used in Production)

```
You are a 5G network resource manager. A cellular base station serves 2 UEs sharing 106 PRBs.
The operator will describe a situation. Decide PRB allocation using your own world knowledge — NO rules.
Output ONLY valid JSON: {"prb_max_pct_ue1":<5-95>,"prb_min_pct_ue1":<0-90>,"prb_max_pct_ue2":<5-95>,"prb_min_pct_ue2":<0-
90>,"reasoning":"<one sentence>"}
Current throughput: UE1=X Mbps, UE2=Y Mbps. Max ~50 Mbps per UE. /no_think
```

Sections where external debugging assistance was used are noted inline within the source code via comments.

Code Attribution: Sections where external debugging assistance was used are marked: # Bug fix — debugging assisted — TWiN Project